

Terminal & GitHub

Command Reference Guide

A complete guide to navigating terminals and pushing code to GitHub

■ CD — Change Directory

cd stands for **Change Directory**. It is one of the most fundamental terminal commands used to navigate between folders (directories) in your file system. Without it, you cannot move from one folder to another inside the terminal.

Command	What it Does
cd folder-name	Move INTO a specific folder
cd ..	Go ONE level UP (parent folder)
cd ~	Jump to the HOME directory
cd /	Go to the ROOT of the file system
cd -	Go back to the PREVIOUS directory
cd ../folder	Go up one level, then into another folder

■ Terminal Navigation Commands

Use these commands to explore and manage your file system before working with Git.

Command	Full Form	Explanation
pwd	Print Working Directory	Shows the full path of your current location
ls	List	Lists all files and folders in current directory
ls -la	List (long + all)	Lists all files including hidden ones with details
mkdir folder	Make Directory	Creates a new folder
rm -rf folder	Remove (recursive, force)	Deletes a folder and all its contents
touch file.txt	Touch	Creates a new empty file
cat file.txt	Concatenate	Displays the contents of a file
clear	Clear	Clears the terminal screen

■ One-Time Git Setup

These commands are run **only once** when you set up Git for the first time on your machine. They configure your identity so every commit is tagged with your name and email.

Command	Explanation
git init	Initializes a new local Git repository in the current folder. Creates a hidden
git config --global user.name "Your Name"	Sets your name globally for all Git commits on this machine.
git config --global user.email "you@email.com"	Sets your email globally for all Git commits.

<code>git config --list</code>	Displays all current Git configuration settings.
--------------------------------	--

■ Core Git Push Workflow

These are the commands you will use **every single day** when working with Git and GitHub. They follow a simple pattern: check → stage → commit → push.

Command	Explanation
<code>git status</code>	Shows which files are modified, staged (ready to commit), or untracked (new files)
<code>git add .</code>	Stages ALL changed files in the current directory. The dot (.) means 'everything here'
<code>git add filename.txt</code>	Stages only a SPECIFIC file. Use when you don't want to commit everything at once
<code>git commit -m "message"</code>	Saves a snapshot of all staged files. The -m flag lets you write a commit message
<code>git remote add origin URL</code>	Links your local repository to a remote GitHub repository using the given URL.
<code>git push origin main</code>	Uploads your local commits to the 'main' branch on GitHub (origin = GitHub).
<code>git push -u origin main</code>	Pushes and sets 'origin main' as the default. Use the -u flag only the FIRST time you push
<code>git push</code>	Shorthand push after default is set with -u. Pushes to the default remote branch.

■ Syncing & Branching Commands

Branches let you work on features separately without affecting the main codebase. Syncing commands keep your local copy up to date with the remote.

Command	Explanation
<code>git pull origin main</code>	Downloads AND merges the latest changes from GitHub into your local branch.
<code>git fetch</code>	Downloads changes from GitHub but does NOT merge them. You review first, merge later
<code>git clone URL</code>	Copies an entire remote repository to your local machine. Creates a new folder.
<code>git branch</code>	Lists all branches in your repository. The active branch shows with an asterisk (*)
<code>git branch branch-name</code>	Creates a new branch with the given name. Does NOT switch to it.
<code>git checkout branch-name</code>	Switches to an existing branch.
<code>git checkout -b branch-name</code>	Creates AND switches to a new branch in a single command.
<code>git merge branch-name</code>	Merges the specified branch into your current active branch.
<code>git branch -d branch-name</code>	Deletes a branch locally (only if it's already been merged).

■ History & Undo Commands

View your commit history and undo mistakes safely.

Command	Explanation
<code>git log</code>	Shows the full commit history with hash, author, date and message.
<code>git log --oneline</code>	Shows a compact one-line-per-commit view of history.
<code>git diff</code>	Shows the exact changes (line by line) between your working files and last commit.
<code>git restore filename</code>	Discards uncommitted changes in a file. Reverts it to the last commit.
<code>git reset HEAD~1</code>	Undoes the last commit but keeps the changes in your files (soft undo).

<code>git stash</code>	Temporarily saves your uncommitted changes so you can switch branches cleanly.
<code>git stash pop</code>	Restores the stashed changes back into your working directory.

■ Step-by-Step: Your First Push to GitHub

Follow these steps in order to push a brand new project to GitHub:

Step 1 — Navigate to your project

```
cd my-project-folder
```

Step 2 — Initialize Git

```
git init
```

Step 3 — Stage all files

```
git add .
```

Step 4 — Create the first commit

```
git commit -m "Initial commit"
```

Step 5 — Link to your GitHub repo

```
git remote add origin https://github.com/username/repo.git
```

Step 6 — Push to GitHub

```
git push -u origin main
```

■ Pro Tips

■ Always run **git status** before committing — it shows exactly what will be included.

■ Write **meaningful commit messages**: 'fix login bug' is better than 'update'.

■ Run **git pull** before pushing when collaborating to avoid merge conflicts.

■ Use **git log --oneline** for a quick clean view of your project history.

■ Use **.gitignore** to exclude files like `node_modules/`, `.env`, or build folders from Git.

■■ **Warning:** Never use **git reset** or **git push --force** on shared/public branches unless you know what you are doing — it can overwrite teammates' work permanently.